

Test Pyramid

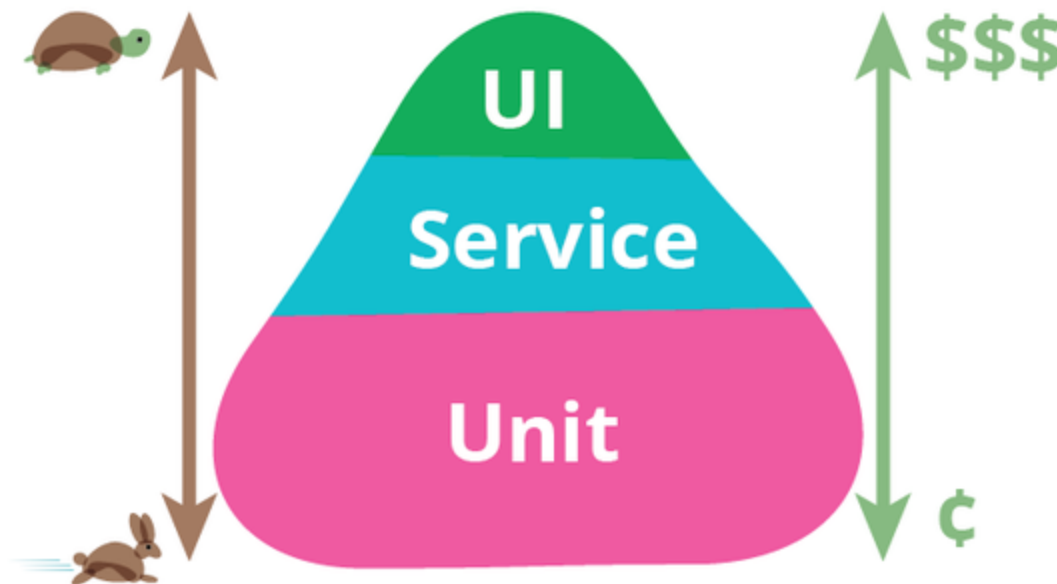
1 May 2012



Martin Fowler

TESTING

The test pyramid is a way of thinking about how different kinds of automated tests should be used to create a balanced portfolio. Its essential point is that you should have many more low-level UnitTests than high level BroadStackTests running through a GUI.



For much of my career test automation meant tests that drove an application through its user-interface. Such tools would often provide the facility to record an interaction with the application and then allow you to play back that interaction, checking that the application returned the same results. Such an approach works well initially. It's easy to

record tests, and the tests can be recorded by people with no knowledge of programming.

But this kind of approach quickly runs into trouble, becoming an ice-cream cone. Testing through the UI like this is slow, increasing build times. Often it requires installed licences for the test automation software, which means it can only be done on particular machines. Usually these cannot easily be run in a “headless” mode, monitored by scripts to put in a proper deployment pipeline.

Most importantly such tests are very brittle. An enhancement to the system can easily end up breaking lots of such tests, which then have to be re-recorded. You can reduce this problem by abandoning record-playback tools, but that makes the tests harder to write. ¹ Even with good practices on writing them, end-to-end tests are more prone to non-determinism problems, which can undermine trust in them. In short, tests that run end-to-end through the UI are: brittle, expensive to write, and time consuming to run. So the pyramid argues that you should do much more automated testing through unit tests than you should through traditional GUI based testing. ²

The pyramid also argues for an intermediate layer of tests that act through a service layer of an application, what I refer to as SubcutaneousTests. These can provide many of the advantages of end-to-end tests but avoid many of the complexities of dealing with UI frameworks. In web applications this would correspond to testing through an API layer while the top UI part of the pyramid would correspond to tests using something like Selenium or Sahi.

The test pyramid comes up a lot in Agile testing circles and while its core message is sound, there is much more to say about building a well-balanced test portfolio. A common problem is that teams conflate the concepts of end-to-end tests, UI tests, and customer facing tests. These are all orthogonal characteristics. For example a rich javascript UI should have most of its UI behavior tested with javascript unit tests using something like Jasmine. A complex set of business rules could have tests captured in a customer-facing form, but run just on the relevant module much as unit tests are.

I always argue that high-level tests are there as a second line of test defense. If you get a failure in a high level test, not just do you have a bug in your functional code, you also have a missing or incorrect unit test. Thus I advise that before fixing a bug exposed by a high level test, you should replicate the bug with a unit test. Then the unit test ensures the bug stays dead.

Further Reading

The Google Testing Blog expands on why you shouldn't rely on end-to-end tests.

Adrian Sutton explains LMAX's experience which shows that end-to-end tests can play a large and valuable role.

Some writers argue that the pyramid isn't a good test distribution, preferring more integration tests and few unit tests. But difference is probably illusory due to different definitions of "unit test".

Acknowledgements

Kevin Dishman gave me the idea of adding the cost and speed arrows to the illustration

Etymology

Most people know about the the Test Pyramid due to Mike Cohn, when he described it in his 2009 book Succeeding with Agile. In the book he refers to it as the "Test Automation Pyramid", but in use it's generally referred to as just the "test pyramid". He originally drew it in conversation with Lisa Crispin in 2003-4 and described it at a scrum gathering in 2004. Jason Huggins independently came up with the same idea around 2006.

Revisions

7 Aug 2016: changed illustration

15 Nov 2017: added etymology

Notes

1: Record-playback tools are almost always a bad idea for any kind of automation, since they resist changeability and obstruct useful abstractions. They are only worth having as a tool to generate fragments of scripts which you can then edit as a proper programming language, in the manner of the venerable Emacs.

2: The pyramid is based on the assumption that broad-stack tests are expensive, slow, and brittle compared to more focused tests, such as unit tests. While this is usually true, there are exceptions. If my high level tests are fast, reliable, and cheap to modify - then lower-level tests aren't needed.